

# CSE 344 – Systems Programming

## HOMEWORK #6 REPORT

*Magic Academy*

|              |                               |
|--------------|-------------------------------|
| Name Surname | FATİH EMRE OĞAN               |
| Student ID   | 230104004090                  |
| Course       | CSE 344 – Systems Programming |
| Homework     | Homework #6 – Magic Academy   |

# Table of Contents

|                                                          |    |
|----------------------------------------------------------|----|
| 1. Introduction.....                                     | 4  |
| 2. System Design .....                                   | 4  |
| 2.1 Single-Threaded Multi-Client Handling .....          | 4  |
| 2.2 Idle Timeout Recalculated Before Every select()..... | 5  |
| 2.3 Rejecting New Connections at Full Capacity.....      | 5  |
| 2.4 Programs and Command-Line Parameters.....            | 5  |
| 3. Data Structures .....                                 | 6  |
| 3.1 Per-Client State (client_t).....                     | 6  |
| 3.2 Spellbook Management.....                            | 6  |
| 3.3 Ingredient List.....                                 | 6  |
| 4. Partial Read / Write Handling .....                   | 7  |
| 5. Signal Handling and Shutdown .....                    | 7  |
| 5.1 Server Ctrl+C Sequence.....                          | 7  |
| 5.2 Wizard and Professor Ctrl+C Sequence.....            | 7  |
| 5.3 Idle Timeout Disconnect.....                         | 8  |
| 5.4 Server Full Rejection .....                          | 8  |
| 6. Build and Usage .....                                 | 8  |
| 7. Test Scenarios .....                                  | 8  |
| 7.1 Scenario 1.....                                      | 8  |
| Server Terminal.....                                     | 9  |
| Wizard hermione Terminal.....                            | 9  |
| Professor dumbledore Terminal.....                       | 10 |
| Duplicate Username (hermione) Terminal.....              | 10 |
| Pre-Enroll Command Terminal.....                         | 11 |
| 7.2 Scenario 2.....                                      | 12 |
| Server Terminal.....                                     | 12 |
| Wizard harry Terminal.....                               | 13 |
| Professor mcgonagall Terminal .....                      | 13 |
| 7.3 Scenario 3.....                                      | 15 |
| Server Terminal.....                                     | 15 |
| Wizard harry Terminal.....                               | 16 |
| Wizard ron Terminal.....                                 | 16 |
| Professor dumbledore Terminal.....                       | 17 |
| Wizard voldemort Terminal (Rejected).....                | 17 |
| Professor snape Terminal (Rejected).....                 | 18 |

|                                                                      |    |
|----------------------------------------------------------------------|----|
| Wizard neville Terminal.....                                         | 18 |
| 7.4 Scenario 4.....                                                  | 20 |
| Server Terminal.....                                                 | 20 |
| Wizard hermione Terminal.....                                        | 21 |
| Wizard harry Terminal (Idle Timeout).....                            | 21 |
| Wizard ron Terminal.....                                             | 22 |
| Professor dumbledore Terminal.....                                   | 22 |
| Professor mcgonagall Terminal .....                                  | 23 |
| 8. Challenges .....                                                  | 24 |
| 8.1 Recomputing the select() Timeout Without sleep() or Threads..... | 24 |
| 8.2 Handling Partial TCP Messages and Clean Disconnection.....       | 24 |

# 1. Introduction

This report describes the design and implementation of the Magic Academy system for Homework #6. The system consists of three separate programs written in C: a single-threaded TCP server (hogwarts), a wizard client (wizard), and a professor client (professor). The server handles multiple simultaneous TCP clients using a single `select()` loop, with no thread-per-client and no use of `sleep()`.

The server loads a list of magical ingredients at startup, manages per-client state including a personal spellbook for each wizard, enforces a maximum number of simultaneous clients, automatically disconnects idle clients after a configurable timeout, and shuts down cleanly on `SIGINT`. All client-server communication uses line-based TCP messages, and partial reads and writes are handled correctly.

The report explains the design decisions behind these mechanisms, documents the data structures used, and presents uncut terminal screenshots for all four mandatory test scenarios together with an explanation of each.

## 2. System Design

### 2.1 Single-Threaded Multi-Client Handling

The server never creates a thread or process per client. Instead, all file descriptors – the listening socket plus every connected client socket – are placed into a single `fd_set` and monitored by one `select()` call inside the main loop. This satisfies the requirement that all TCP I/O be handled inside one `select()` loop in a single thread.

Each iteration of the main loop performs four ordered steps:

1. Idle timeout sweep: every connected client is checked, and any client idle longer than the timeout is disconnected before `select()` is called again.
2. Build the `fd_set`: the listening socket and all client sockets are added, and the smallest remaining time until the next possible timeout is computed.
3. Accept new connections: if the listening socket is readable, a new connection is accepted (and immediately rejected if the server is at capacity).
4. Service existing clients: every client socket that is readable is read and its complete command lines are processed.

Because a single descriptor set drives the whole loop, the server scales to at least the maximum number of clients given by the `-n` option without any per-client thread.

## 2.2 Idle Timeout Recalculated Before Every select()

Idle timeout is implemented without `sleep()` and without threads. Each client record stores a `last_active` timestamp obtained with `time()`, updated whenever a command is received from that client.

Before every `select()` call the server recomputes the timeout as follows:

- It iterates over all connected clients and computes the elapsed time since each client's last activity.
- Any client whose elapsed time has reached the configured timeout is sent `TIMEOUT DISCONNECT`, the event is logged, and the client is removed.
- For the remaining clients, the earliest future expiry time is found, and the difference between that time and the current time is passed as the timeout argument (`struct timeval`) to `select()`.

This guarantees that `select()` never blocks past the moment the next client would time out, so idle clients are disconnected promptly without busy-waiting or sleeping.

## 2.3 Rejecting New Connections at Full Capacity

The server tracks the current number of connected clients. When a new connection arrives while the server is already at the maximum (`-n`), the connection is still accepted with `accept()` so the client receives a meaningful response, then the server immediately sends `ERR HOGWARTS_FULL`, logs a `REJECTED` line with the file descriptor and IP address, and closes the descriptor without adding it to the client list.

Because the rejected descriptor is closed and never stored, a slot is recovered as soon as any existing client disconnects (via `APPARATE`, timeout, or hangup), allowing a subsequent client to connect successfully. This slot recovery behaviour is demonstrated in Test Scenario 3.

## 2.4 Programs and Command-Line Parameters

Three programs are produced. The server is started with five parameters and the clients with three positional arguments:

```
./hogwarts -p <tcp_port> -s <ingredients.txt> -l <logfile> -n <max_clients> -t <timeout>
./wizard <server_ip> <tcp_port> <username>
./professor <server_ip> <tcp_port> <username>
```

Invalid or missing parameters cause a usage message on `stderr` and a non-zero exit code. The port must be at least 1024, `max_clients` at least 1, and timeout at least 1.

## 3. Data Structures

### 3.1 Per-Client State (client\_t)

The server keeps one `client_t` record for every connected client. The structure follows the minimum required structure from the assignment and adds a few fields needed for correct line buffering and spellbook management:

```
typedef struct {
    int fd;                /* client socket */
    char username[32];      /* registered name */
    char type[16];          /* WIZARD or PROFESSOR */
    char line_buf[512+2];   /* partial read buffer */
    int line_len;           /* bytes currently buffered */
    int enrolled;          /* 1 after a valid ENROLL */
    time_t last_active;     /* timestamp of last command */
    spellbook_entry_t spellbook[256]; /* WIZARD only */
    int spellbook_count;    /* entries in use */
} client_t;
```

Client records are allocated dynamically with `calloc()` and stored in a pointer table. When a client disconnects for any reason (APPARATE, timeout, or hangup), its socket is closed, its memory is freed, and its slot in the table is cleared, so no file descriptor or memory leaks after a disconnect.

### 3.2 Spellbook Management

Each wizard has its own spellbook stored inside its `client_t` record as an array of `spellbook_entry_t` (an ingredient name and a quantity). A BREW command increases the quantity in both the global stock and the wizard's spellbook, creating a new spellbook entry if the ingredient is not present yet. A CONSUME command requires that both the global stock and the wizard's spellbook hold at least the requested quantity; otherwise ERR

INSUFFICIENT\_INGREDIENTS is returned. The spellbook is not persisted: it lives only inside the client record and is lost when the client disconnects.

### 3.3 Ingredient List

The global ingredient list is a fixed-size array of `ingredient_t` (name and quantity). It is loaded once at startup from the file given by `-s`. Each line must contain an ingredient name (uppercase letters, digits, or underscore, up to 16 characters) and a positive integer quantity; lines with an invalid format are skipped silently. Global quantities are updated by BREW and CONSUME and are always reported as non-negative integers.

## 4. Partial Read / Write Handling

TCP does not preserve message boundaries, so a single `read()` may return part of a line, several lines, or a line split across multiple reads. The server therefore keeps a per-client line buffer (`line_buf`) and a length counter (`line_len`).

On each readable event the server reads available bytes into the buffer after the existing data, then scans for newline characters. Every complete line (terminated by a newline) is processed as one command. Any bytes after the last newline are an incomplete line; they are moved to the front of the buffer and kept until the rest of the line arrives in a later read. A line longer than the maximum length is answered with `ERR TOOLONG`.

Partial writes are handled by a helper that loops on `write()` until all bytes of a response have been sent, restarting on `EINTR`. This guarantees that responses are delivered completely even if the kernel accepts only part of the buffer in one call. The clients use the same line-buffering technique when reading server responses, so split or merged server messages are also handled correctly.

## 5. Signal Handling and Shutdown

### 5.1 Server Ctrl+C Sequence

5. The user presses Ctrl+C; the `SIGINT` handler runs and performs only an async-signal-safe operation – it sets a volatile `sig_atomic_t` flag.
6. The main `select()` loop detects the flag at the next safe point and leaves the loop, entering shutdown mode.
7. `SERVER SHUTDOWN` is sent to every connected TCP client.
8. All client sockets and the listening socket are closed, and every client record is freed.
9. `SHUTDOWN signal=SIGINT` and `CLEANUP_DONE clients=0` are written to the log.
10. The process exits cleanly with no zombie processes and no leaked file descriptors.

### 5.2 Wizard and Professor Ctrl+C Sequence

11. The user presses Ctrl+C in a client terminal; the client's `SIGINT` handler sets a volatile `sig_atomic_t` flag.
12. The main loop detects the flag and sends `APPARATE` to the server, then waits for the reply.
13. The server responds with `OK APPARATE` and logs `CLIENT_DISCONNECTED reason=APPARATE`.

14. The client prints DISCONNECTED reason=APPARATE, closes the TCP connection, and exits cleanly.

### 5.3 Idle Timeout Disconnect

When the server detects an idle client it sends TIMEOUT DISCONNECT, logs TIMEOUT username=X fd=N elapsed=Ns and CLIENT\_DISCONNECTED reason=timeout, removes the client from the fd\_set, and frees its state. The client prints a timeout message and exits cleanly.

### 5.4 Server Full Rejection

When a new client connects while the server is at capacity, the server accepts the connection, immediately sends ERR HOGWARTS\_FULL, logs REJECTED fd=N ip=X reason=HOGWARTS\_FULL, and closes the descriptor without adding it to the client list.

## 6. Build and Usage

All three programs are built with a single Makefile using the required compiler flags. The build produces zero warnings with -Wall -Wextra.

```
make      # builds hogwarts, wizard, professor
make clean # removes all binaries and object files
```

Compiler invocation used by the Makefile:

```
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L
```

A note on timeout values: in Test Scenarios 1, 2 and 3 the idle timeout was set to a longer value (90 seconds) so that the multi-terminal steps could be performed comfortably by hand. The timeout is a configurable command-line parameter and the underlying mechanism is identical regardless of its value. Test Scenario 4, which specifically demonstrates the idle timeout, uses the short value (10 seconds) as intended.

## 7. Test Scenarios

Each scenario was run with multiple terminals open simultaneously. Every terminal was screenshotted separately, and every screenshot is uncut, showing the full terminal from the command prompt through program exit. A fresh server was started for each scenario.

### 7.1 Scenario 1

Command:

```
./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 90
```

Steps performed: wizard hermione and professor dumbledore connect; a second wizard with the same username hermione is rejected with ERR ENROLL name\_taken; a third client connects and sends a command before ENROLL and receives ERR NOT\_ENROLLED; hermione sends FLY BROOMSTICK (ERR UNKNOWN); dumbledore sends BREW and CONSUME (ERR



UNAUTHORIZED); hermione sends INSPECT and SCROLL (ERR UNAUTHORIZED); hermione and dumbledore exit with Ctrl+C.

## Server Terminal

```
fatih@fatih: ~/Desktop/230104004090_hw6
File Edit View Search Terminal Help
fatih@fatih:~/Desktop/230104004090_hw6$ make
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o hogwarts hogwarts.c common.c
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o wizard wizard.c common.c
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o professor professor.c common.c
fatih@fatih:~/Desktop/230104004090_hw6$ ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 90
Hogwarts is ready. Port: 5000 | Max Clients: 4 | Timeout: 90s
[2026-05-17 11:17:51] SERVER_STARTED port=5000 max_clients=4 timeout=90 ingredients=5
[2026-05-17 11:18:01] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:18:01] ENROLL username=hermione type=WIZARD fd=5
[2026-05-17 11:18:07] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-17 11:18:07] ENROLL username=dumbledore type=PROFESSOR fd=6
[2026-05-17 11:18:18] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-17 11:18:26] CLIENT_DISCONNECTED username=(unenrolled) reason=hangup
[2026-05-17 11:18:32] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-17 11:18:46] CLIENT_DISCONNECTED username=(unenrolled) reason=hangup
[2026-05-17 11:19:23] CLIENT_DISCONNECTED username=hermione reason=APPARATE
[2026-05-17 11:19:26] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
^C[2026-05-17 11:19:32] SHUTDOWN signal=SIGINT
[2026-05-17 11:19:32] CLEANUP_DONE clients=0
fatih@fatih:~/Desktop/230104004090_hw6$
```

## Wizard hermione Terminal

```
fatih@fatih: ~/Desktop/230104004090_hw6
fatih@fatih:~/Desktop/230104004090_hw6$ ./wizard 127.0.0.1 5000 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5000
[WIZARD hermione] RECEIVED OK ENROLL hermione
FLY BROOMSTICK
[WIZARD hermione] SENT FLY BROOMSTICK
[WIZARD hermione] RECEIVED ERR UNKNOWN FLY BROOMSTICK
INSPECT MOONSTONE
[WIZARD hermione] SENT INSPECT MOONSTONE
[WIZARD hermione] RECEIVED ERR UNAUTHORIZED
SCROLL
[WIZARD hermione] SENT SCROLL
[WIZARD hermione] RECEIVED ERR UNAUTHORIZED
^C[WIZARD hermione] SENT APPARATE
[WIZARD hermione] RECEIVED OK APPARATE
[WIZARD hermione] DISCONNECTED reason=APPARATE
fatih@fatih:~/Desktop/230104004090_hw6$
```

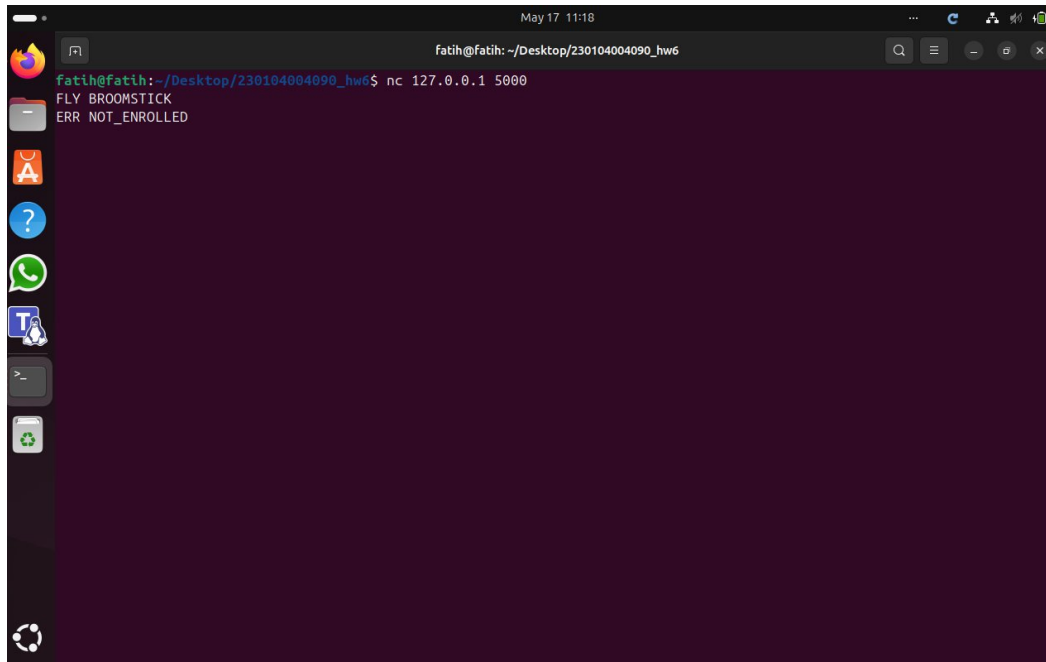
## Professor dumbledore Terminal

```
fatih@fatih: ~/Desktop/230104004090_hw6
fatih@fatih:~/Desktop/230104004090_hw6$ ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5000
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
BREW MOONSTONE 5
[PROFESSOR dumbledore] SENT BREW MOONSTONE 5
[PROFESSOR dumbledore] RECEIVED ERR UNAUTHORIZED
CONSUME MOONSTONE 5
[PROFESSOR dumbledore] SENT CONSUME MOONSTONE 5
[PROFESSOR dumbledore] RECEIVED ERR UNAUTHORIZED
^C[PROFESSOR dumbledore] SENT APPARATE
[PROFESSOR dumbledore] RECEIVED OK APPARATE
[PROFESSOR dumbledore] DISCONNECTED reason=APPARATE
fatih@fatih:~/Desktop/230104004090_hw6$
```

## Duplicate Username (hermione) Terminal

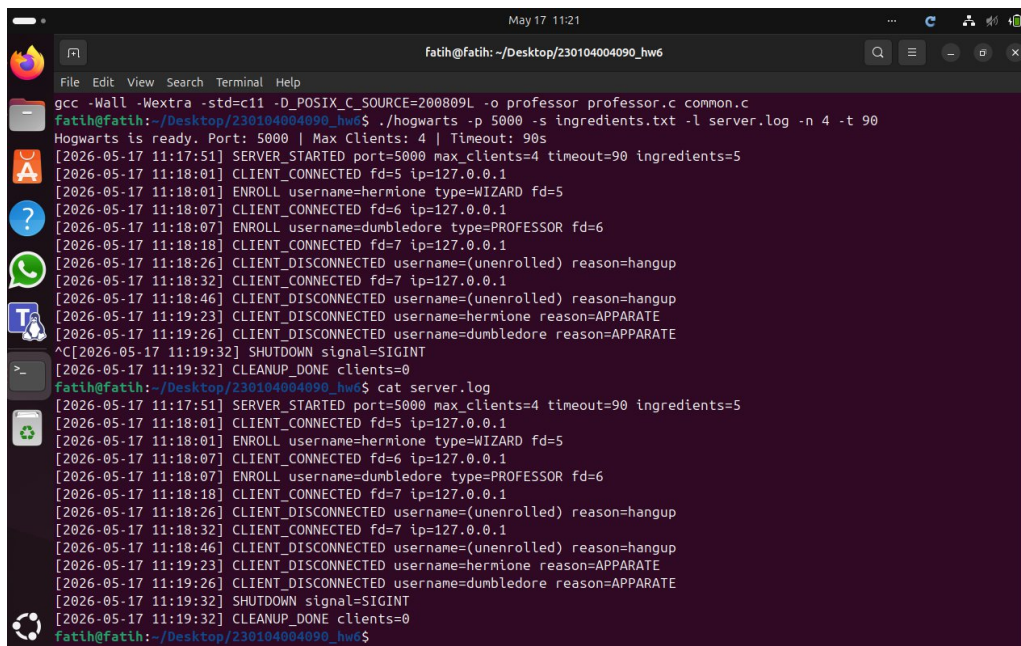
```
fatih@fatih: ~/Desktop/230104004090_hw6
fatih@fatih:~/Desktop/230104004090_hw6$ ./wizard 127.0.0.1 5000 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5000
[WIZARD hermione] RECEIVED ERR ENROLL name_taken
```

## Pre-Enroll Command Terminal



```
fatih@fatih: ~/Desktop/230104004090_hw6
fatih@fatih:~/Desktop/230104004090_hw6$ nc 127.0.0.1 5000
FLY BROOMSTICK
ERR NOT_ENROLLED
```

*Explanation: The screenshots show that duplicate usernames are rejected, commands sent before ENROLL are rejected, unknown commands and unauthorized commands return the correct error messages, and both clients exit cleanly with APPARATE. The server log records every event in order. Note: Timeout was set to 90 s (instead of 30 s) only for manual convenience; the -t value is configurable and does not change the mechanism.*



```
fatih@fatih: ~/Desktop/230104004090_hw6
File Edit View Search Terminal Help
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o professor professor.c common.c
fatih@fatih:~/Desktop/230104004090_hw6$ ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 90
Hogwarts is ready. Port: 5000 | Max Clients: 4 | Timeout: 90s
[2026-05-17 11:17:51] SERVER_STARTED port=5000 max_clients=4 timeout=90 ingredients=5
[2026-05-17 11:18:01] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:18:01] ENROLL username=hermione type=WIZARD fd=5
[2026-05-17 11:18:07] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-17 11:18:07] ENROLL username=dumbledore type=PROFESSOR fd=6
[2026-05-17 11:18:18] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-17 11:18:26] CLIENT_DISCONNECTED username=(unenrolled) reason=hangup
[2026-05-17 11:18:32] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-17 11:18:46] CLIENT_DISCONNECTED username=(unenrolled) reason=hangup
[2026-05-17 11:19:23] CLIENT_DISCONNECTED username=hermione reason=APPARATE
[2026-05-17 11:19:26] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
^C[2026-05-17 11:19:32] SHUTDOWN signal=SIGINT
[2026-05-17 11:19:32] CLEANUP_DONE clients=0
fatih@fatih:~/Desktop/230104004090_hw6$ cat server.log
[2026-05-17 11:17:51] SERVER_STARTED port=5000 max_clients=4 timeout=90 ingredients=5
[2026-05-17 11:18:01] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:18:01] ENROLL username=hermione type=WIZARD fd=5
[2026-05-17 11:18:07] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-17 11:18:07] ENROLL username=dumbledore type=PROFESSOR fd=6
[2026-05-17 11:18:18] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-17 11:18:26] CLIENT_DISCONNECTED username=(unenrolled) reason=hangup
[2026-05-17 11:18:32] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-17 11:18:46] CLIENT_DISCONNECTED username=(unenrolled) reason=hangup
[2026-05-17 11:19:23] CLIENT_DISCONNECTED username=hermione reason=APPARATE
[2026-05-17 11:19:26] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-17 11:19:32] SHUTDOWN signal=SIGINT
[2026-05-17 11:19:32] CLEANUP_DONE clients=0
fatih@fatih:~/Desktop/230104004090_hw6$
```

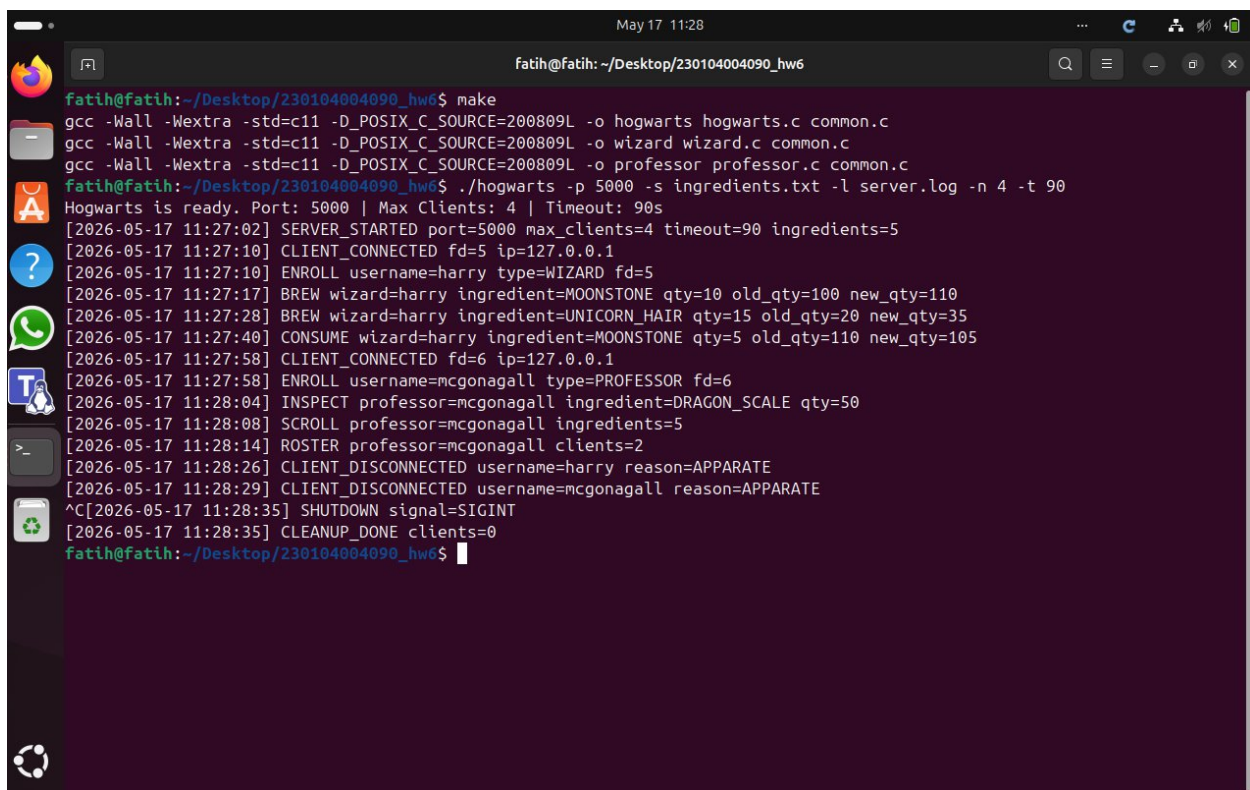
## 7.2 Scenario 2

Command:

```
./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 90
```

Steps performed: wizard harry and professor mcgonagall connect. harry runs BREW MOONSTONE 10, SPELLBOOK, BREW UNICORN\_HAIR 15, SPELLBOOK, CONSUME MOONSTONE 5, CONSUME UNICORN\_HAIR 20 (insufficient), BREW BUTTERBEER 5 (unknown ingredient). mcgonagall runs INSPECT DRAGON\_SCALE, SCROLL, ROSTER, INSPECT BUTTERBEER (unknown ingredient). Both exit with Ctrl+C.

### Server Terminal



```
fatih@fatih: ~/Desktop/230104004090_hw6
fatih@fatih:~/Desktop/230104004090_hw6$ make
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o hogwarts hogwarts.c common.c
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o wizard wizard.c common.c
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o professor professor.c common.c
fatih@fatih:~/Desktop/230104004090_hw6$ ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 90
Hogwarts is ready. Port: 5000 | Max Clients: 4 | Timeout: 90s
[2026-05-17 11:27:02] SERVER_STARTED port=5000 max_clients=4 timeout=90 ingredients=5
[2026-05-17 11:27:10] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:27:10] ENROLL username=harry type=WIZARD fd=5
[2026-05-17 11:27:17] BREW wizard=harry ingredient=MOONSTONE qty=10 old_qty=100 new_qty=110
[2026-05-17 11:27:28] BREW wizard=harry ingredient=UNICORN_HAIR qty=15 old_qty=20 new_qty=35
[2026-05-17 11:27:40] CONSUME wizard=harry ingredient=MOONSTONE qty=5 old_qty=110 new_qty=105
[2026-05-17 11:27:58] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-17 11:27:58] ENROLL username=mcgonagall type=PROFESSOR fd=6
[2026-05-17 11:28:04] INSPECT professor=mcgonagall ingredient=DRAGON_SCALE qty=50
[2026-05-17 11:28:08] SCROLL professor=mcgonagall ingredients=5
[2026-05-17 11:28:14] ROSTER professor=mcgonagall clients=2
[2026-05-17 11:28:26] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-17 11:28:29] CLIENT_DISCONNECTED username=mcgonagall reason=APPARATE
^C[2026-05-17 11:28:35] SHUTDOWN signal=SIGINT
[2026-05-17 11:28:35] CLEANUP_DONE clients=0
fatih@fatih:~/Desktop/230104004090_hw6$
```



## Wizard harry Terminal

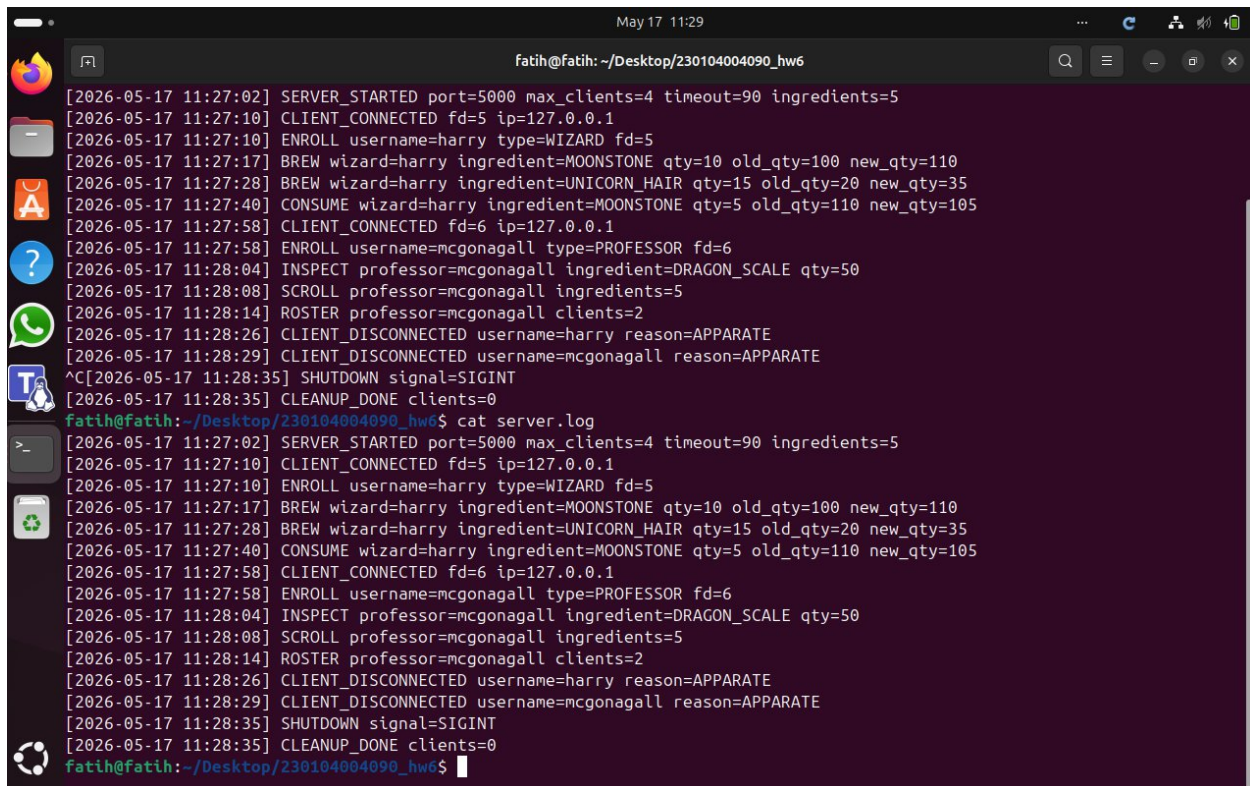
```
fatih@fatih: ~/Desktop/230104004090_hw6
fatih@fatih:~/Desktop/230104004090_hw6$ ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5000
[WIZARD harry] RECEIVED OK ENROLL harry
BREW MOONSTONE 10
[WIZARD harry] SENT BREW MOONSTONE 10
[WIZARD harry] RECEIVED OK BREW MOONSTONE 10 110
SPELLBOOK
[WIZARD harry] SENT SPELLBOOK
[WIZARD harry] RECEIVED OK SPELLBOOK MOONSTONE:10
BREW UNICORN_HAIR 15
[WIZARD harry] SENT BREW UNICORN_HAIR 15
[WIZARD harry] RECEIVED OK BREW UNICORN_HAIR 15 35
SPELLBOOK
[WIZARD harry] SENT SPELLBOOK
[WIZARD harry] RECEIVED OK SPELLBOOK MOONSTONE:10,UNICORN_HAIR:15
CONSUME MOONSTONE 5
[WIZARD harry] SENT CONSUME MOONSTONE 5
[WIZARD harry] RECEIVED OK CONSUME MOONSTONE 5 105
CONSUME UNICORN_HAIR 20
[WIZARD harry] SENT CONSUME UNICORN_HAIR 20
[WIZARD harry] RECEIVED ERR INSUFFICIENT_INGREDIENTS
BREW BUTTERBEER 5
[WIZARD harry] SENT BREW BUTTERBEER 5
[WIZARD harry] RECEIVED ERR UNKNOWN_INGREDIENT
^C[WIZARD harry] SENT APPARATE
[WIZARD harry] RECEIVED OK APPARATE
[WIZARD harry] DISCONNECTED reason=APPARATE
fatih@fatih:~/Desktop/230104004090_hw6$
```

## Professor mcgonagall Terminal

```
fatih@fatih: ~/Desktop/230104004090_hw6
fatih@fatih:~/Desktop/230104004090_hw6$ ./professor 127.0.0.1 5000 mcgonagall
[PROFESSOR mcgonagall] CONNECTED server=127.0.0.1:5000
[PROFESSOR mcgonagall] RECEIVED OK ENROLL mcgonagall
INSPECT DRAGON_SCALE
[PROFESSOR mcgonagall] SENT INSPECT DRAGON_SCALE
[PROFESSOR mcgonagall] RECEIVED OK INSPECT DRAGON_SCALE 50
SCROLL
[PROFESSOR mcgonagall] SENT SCROLL
[PROFESSOR mcgonagall] RECEIVED OK SCROLL MOONSTONE:105,DRAGON_SCALE:50,PHOENIX_FEATHER:30,MANDRAKE_ROOT:75,UNICORN_HAIR:35
ROSTER
[PROFESSOR mcgonagall] SENT ROSTER
[PROFESSOR mcgonagall] RECEIVED OK ROSTER harry,mcgonagall
INSPECT BUTTERBEER
[PROFESSOR mcgonagall] SENT INSPECT BUTTERBEER
[PROFESSOR mcgonagall] RECEIVED ERR UNKNOWN_INGREDIENT
^C[PROFESSOR mcgonagall] SENT APPARATE
[PROFESSOR mcgonagall] RECEIVED OK APPARATE
[PROFESSOR mcgonagall] DISCONNECTED reason=APPARATE
fatih@fatih:~/Desktop/230104004090_hw6$
```

*Explanation: The screenshots show that BREW and CONSUME correctly update both the global stock and the wizard's spellbook, that an over-consumption is rejected with ERR INSUFFICIENT\_INGREDIENTS, that an unknown ingredient is rejected with ERR UNKNOWN\_INGREDIENT, and that the professor queries (INSPECT, SCROLL, ROSTER) return consistent values matching the server log.*

*Note: Timeout was set to 90 s only for manual convenience; this does not affect the protocol behaviour shown here.*



The screenshot shows a terminal window titled "fatih@fatih: ~/Desktop/230104004090\_hw6" with a timestamp of "May 17 11:29". The terminal displays a log of server events. The first part of the log shows the server starting, a wizard (harry) connecting and performing BREW and CONSUME actions, and a professor (mcgonagall) connecting and performing INSPECT, SCROLL, and ROSTER actions. The second part of the log is the output of the command "cat server.log", which repeats the same sequence of events. The terminal window has a dark background and a sidebar with various application icons on the left.

```
fatih@fatih: ~/Desktop/230104004090_hw6
[2026-05-17 11:27:02] SERVER_STARTED port=5000 max_clients=4 timeout=90 ingredients=5
[2026-05-17 11:27:10] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:27:10] ENROLL username=harry type=WIZARD fd=5
[2026-05-17 11:27:17] BREW wizard=harry ingredient=MOONSTONE qty=10 old_qty=100 new_qty=110
[2026-05-17 11:27:28] BREW wizard=harry ingredient=UNICORN_HAIR qty=15 old_qty=20 new_qty=35
[2026-05-17 11:27:40] CONSUME wizard=harry ingredient=MOONSTONE qty=5 old_qty=110 new_qty=105
[2026-05-17 11:27:58] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-17 11:27:58] ENROLL username=mcgonagall type=PROFESSOR fd=6
[2026-05-17 11:28:04] INSPECT professor=mcgonagall ingredient=DRAGON_SCALE qty=50
[2026-05-17 11:28:08] SCROLL professor=mcgonagall ingredients=5
[2026-05-17 11:28:14] ROSTER professor=mcgonagall clients=2
[2026-05-17 11:28:26] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-17 11:28:29] CLIENT_DISCONNECTED username=mcgonagall reason=APPARATE
^C[2026-05-17 11:28:35] SHUTDOWN signal=SIGINT
[2026-05-17 11:28:35] CLEANUP_DONE clients=0
fatih@fatih:~/Desktop/230104004090_hw6$ cat server.log
[2026-05-17 11:27:02] SERVER_STARTED port=5000 max_clients=4 timeout=90 ingredients=5
[2026-05-17 11:27:10] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:27:10] ENROLL username=harry type=WIZARD fd=5
[2026-05-17 11:27:17] BREW wizard=harry ingredient=MOONSTONE qty=10 old_qty=100 new_qty=110
[2026-05-17 11:27:28] BREW wizard=harry ingredient=UNICORN_HAIR qty=15 old_qty=20 new_qty=35
[2026-05-17 11:27:40] CONSUME wizard=harry ingredient=MOONSTONE qty=5 old_qty=110 new_qty=105
[2026-05-17 11:27:58] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-17 11:27:58] ENROLL username=mcgonagall type=PROFESSOR fd=6
[2026-05-17 11:28:04] INSPECT professor=mcgonagall ingredient=DRAGON_SCALE qty=50
[2026-05-17 11:28:08] SCROLL professor=mcgonagall ingredients=5
[2026-05-17 11:28:14] ROSTER professor=mcgonagall clients=2
[2026-05-17 11:28:26] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-17 11:28:29] CLIENT_DISCONNECTED username=mcgonagall reason=APPARATE
[2026-05-17 11:28:35] SHUTDOWN signal=SIGINT
[2026-05-17 11:28:35] CLEANUP_DONE clients=0
fatih@fatih:~/Desktop/230104004090_hw6$
```

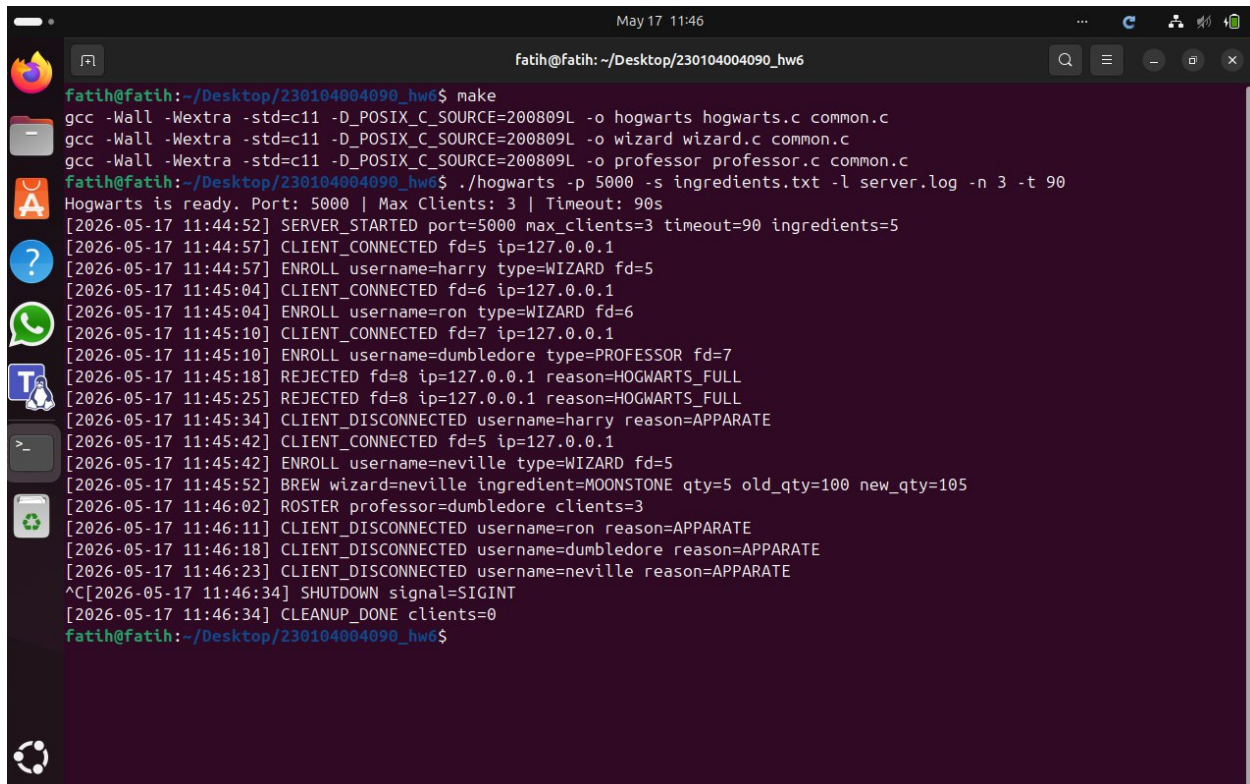
## 7.3 Scenario 3

Command:

```
./hogwarts -p 5000 -s ingredients.txt -l server.log -n 3 -t 90
```

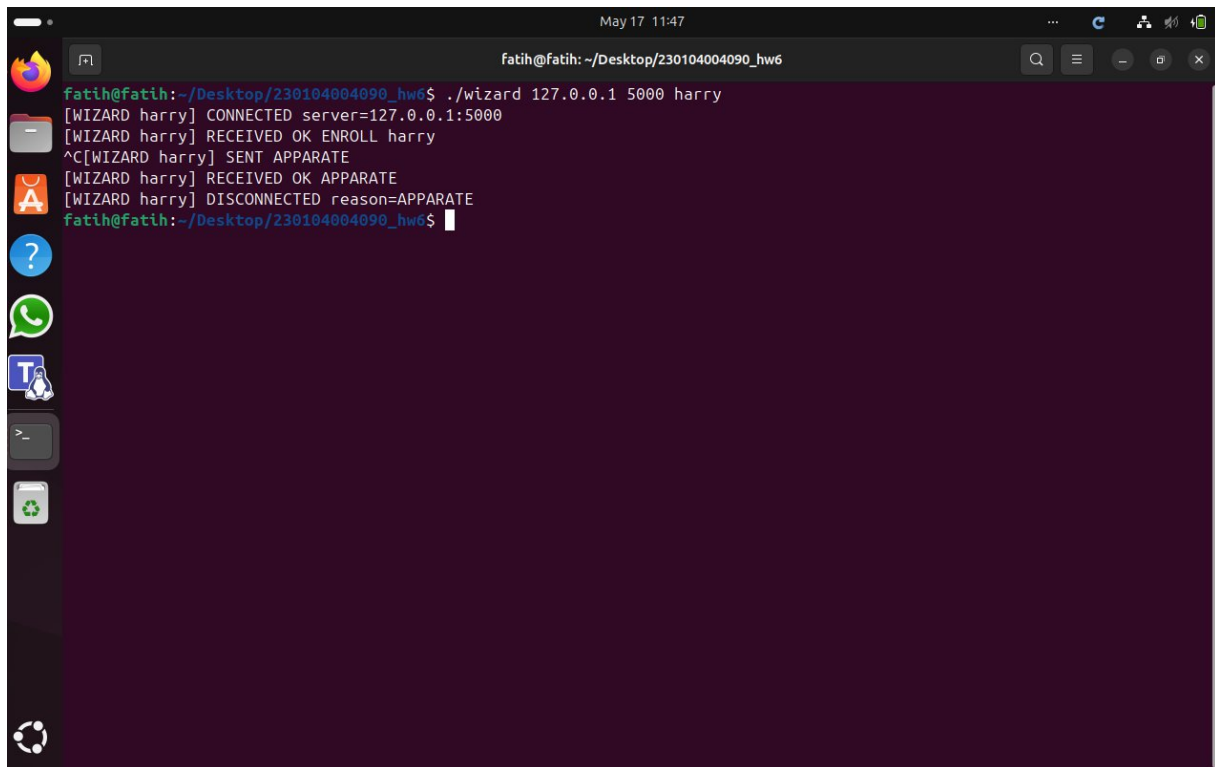
Steps performed: wizard harry, wizard ron, and professor dumbledore connect, filling the server to capacity (n=3). A fourth client (wizard voldemort) and a fifth client (professor snape) are both rejected with ERR HOGWARTS\_FULL. harry exits with Ctrl+C, freeing a slot. wizard neville then connects successfully and runs BREW MOONSTONE 5. dumbledore runs ROSTER. ron, dumbledore, and neville exit with Ctrl+C.

### Server Terminal



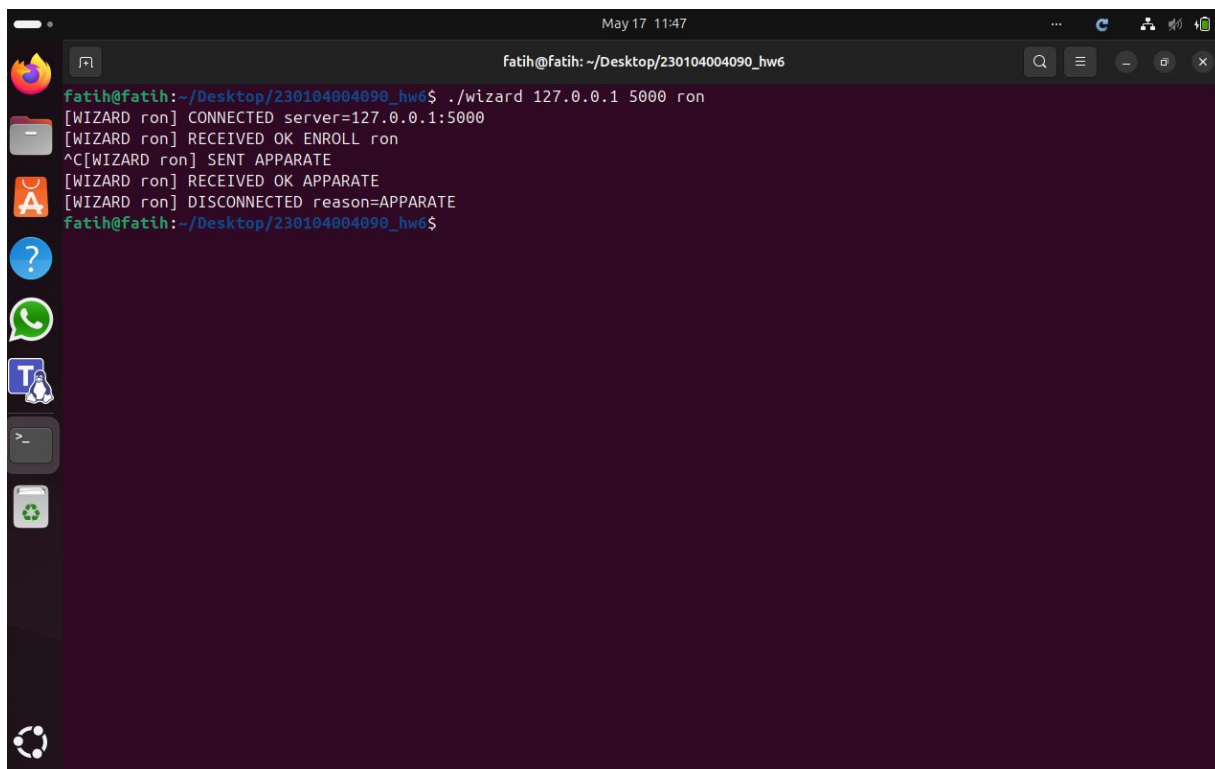
```
fatih@fatih: ~/Desktop/230104004090_hw6
fatih@fatih:~/Desktop/230104004090_hw6$ make
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o hogwarts hogwarts.c common.c
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o wizard wizard.c common.c
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o professor professor.c common.c
fatih@fatih:~/Desktop/230104004090_hw6$ ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 3 -t 90
Hogwarts is ready. Port: 5000 | Max Clients: 3 | Timeout: 90s
[2026-05-17 11:44:52] SERVER_STARTED port=5000 max_clients=3 timeout=90 ingredients=5
[2026-05-17 11:44:57] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:44:57] ENROLL username=harry type=WIZARD fd=5
[2026-05-17 11:45:04] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-17 11:45:04] ENROLL username=ron type=WIZARD fd=6
[2026-05-17 11:45:10] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-17 11:45:10] ENROLL username=dumbledore type=PROFESSOR fd=7
[2026-05-17 11:45:18] REJECTED fd=8 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-17 11:45:25] REJECTED fd=8 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-17 11:45:34] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-17 11:45:42] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:45:42] ENROLL username=neville type=WIZARD fd=5
[2026-05-17 11:45:52] BREW wizard=neville ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-17 11:46:02] ROSTER professor=dumbledore clients=3
[2026-05-17 11:46:11] CLIENT_DISCONNECTED username=ron reason=APPARATE
[2026-05-17 11:46:18] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-17 11:46:23] CLIENT_DISCONNECTED username=neville reason=APPARATE
^C[2026-05-17 11:46:34] SHUTDOWN signal=SIGINT
[2026-05-17 11:46:34] CLEANUP_DONE clients=0
fatih@fatih:~/Desktop/230104004090_hw6$
```

## Wizard harry Terminal



```
fatih@fatih:~/Desktop/230104004090_hw6$ ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5000
[WIZARD harry] RECEIVED OK ENROLL harry
^C[WIZARD harry] SENT APPARATE
[WIZARD harry] RECEIVED OK APPARATE
[WIZARD harry] DISCONNECTED reason=APPARATE
fatih@fatih:~/Desktop/230104004090_hw6$
```

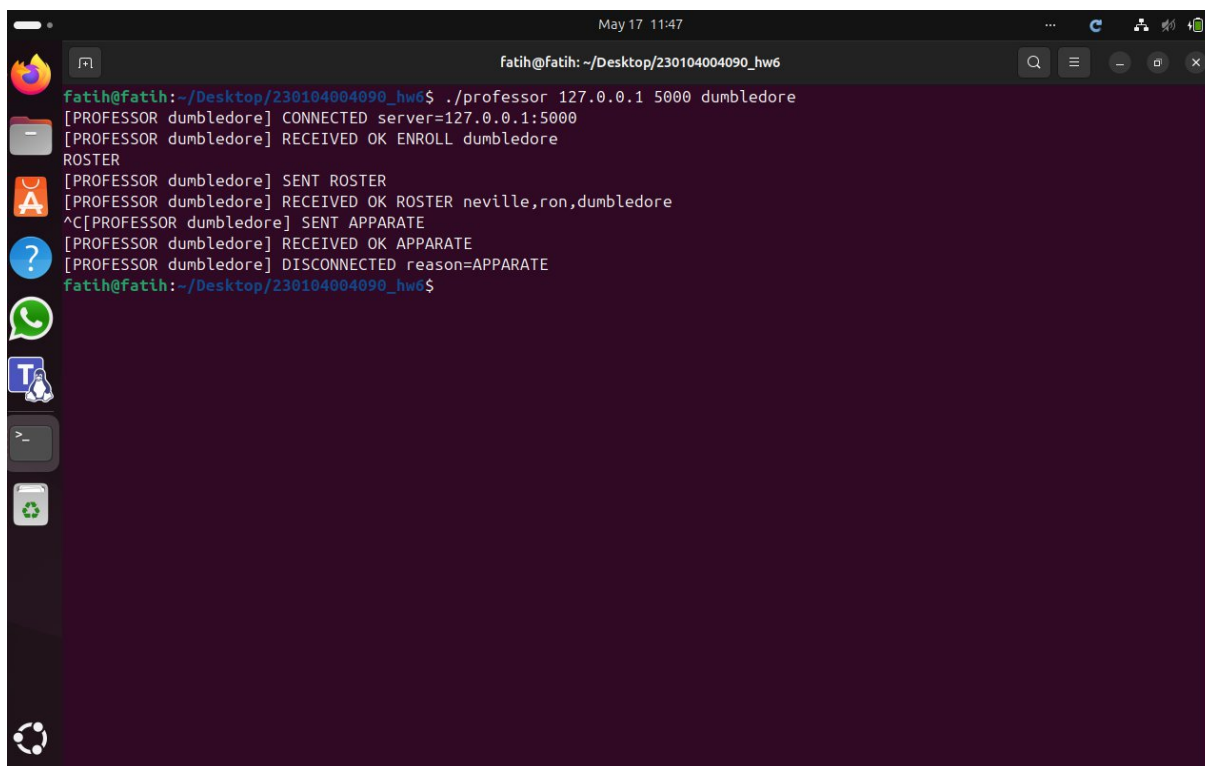
## Wizard ron Terminal



```
fatih@fatih:~/Desktop/230104004090_hw6$ ./wizard 127.0.0.1 5000 ron
[WIZARD ron] CONNECTED server=127.0.0.1:5000
[WIZARD ron] RECEIVED OK ENROLL ron
^C[WIZARD ron] SENT APPARATE
[WIZARD ron] RECEIVED OK APPARATE
[WIZARD ron] DISCONNECTED reason=APPARATE
fatih@fatih:~/Desktop/230104004090_hw6$
```

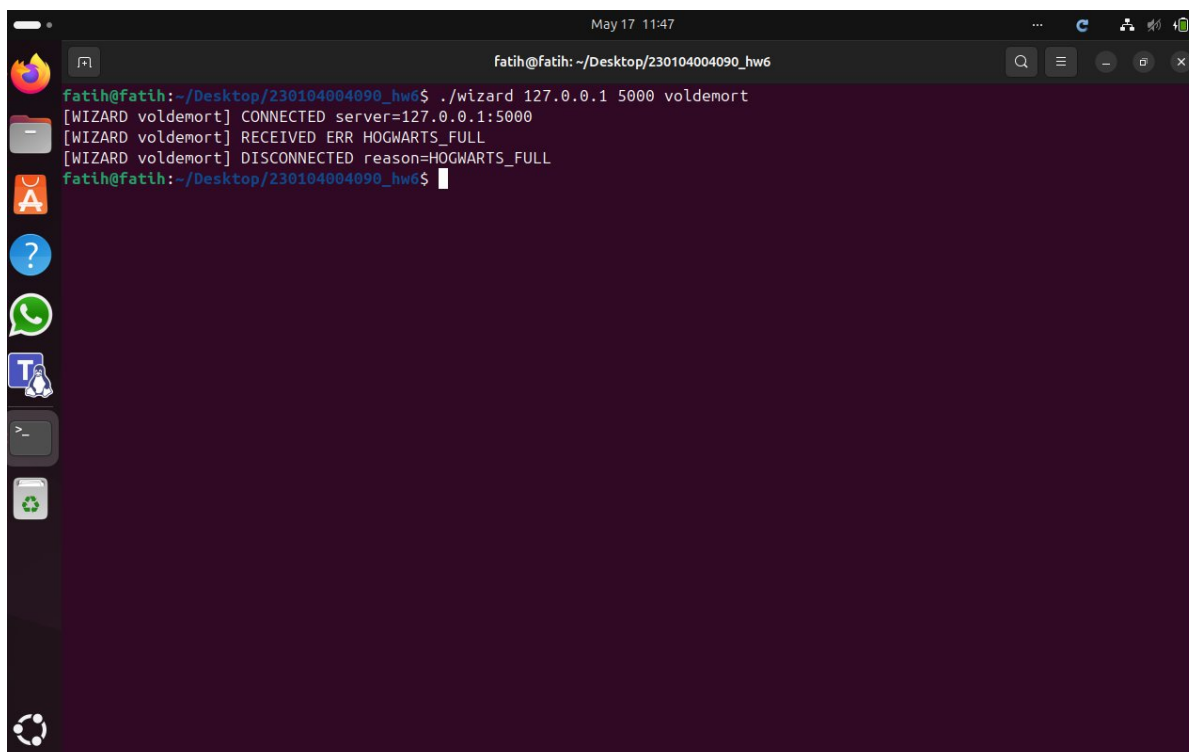


## Professor dumbledore Terminal



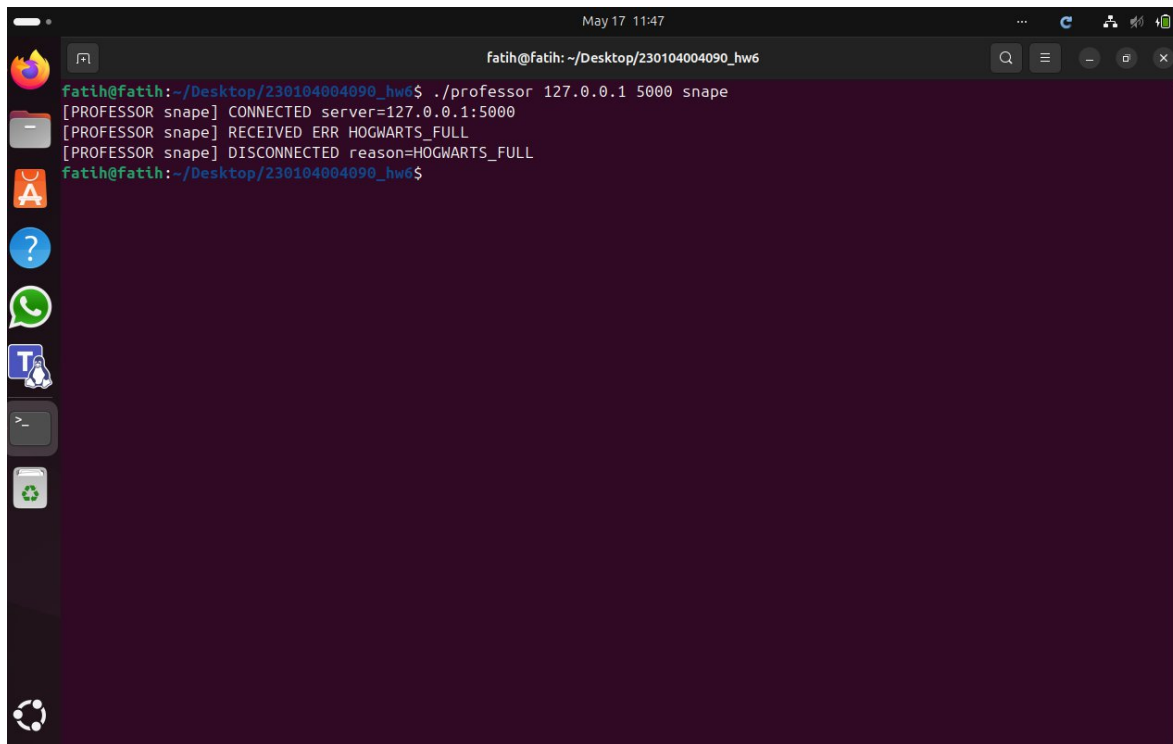
```
fatih@fatih:~/Desktop/230104004090_hw6$ ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5000
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
ROSTER
[PROFESSOR dumbledore] SENT ROSTER
[PROFESSOR dumbledore] RECEIVED OK ROSTER neville,ron,dumbledore
^C[PROFESSOR dumbledore] SENT APPARATE
[PROFESSOR dumbledore] RECEIVED OK APPARATE
[PROFESSOR dumbledore] DISCONNECTED reason=APPARATE
fatih@fatih:~/Desktop/230104004090_hw6$
```

## Wizard voldemort Terminal (Rejected)



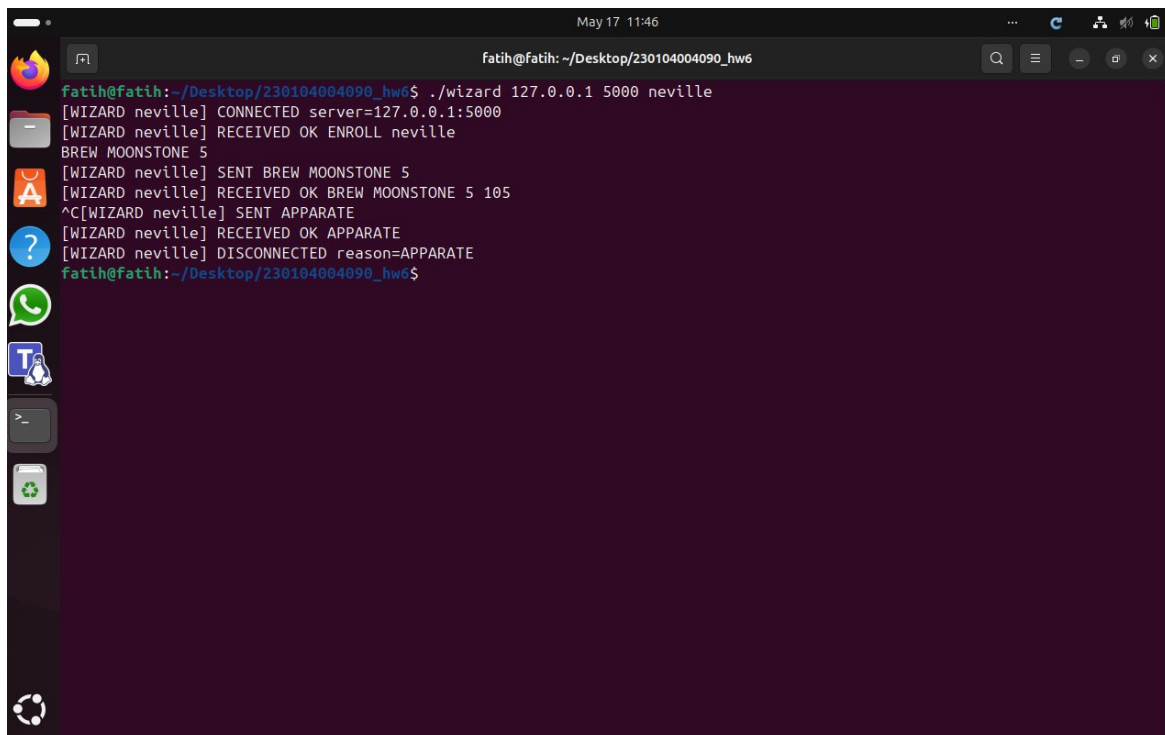
```
fatih@fatih:~/Desktop/230104004090_hw6$ ./wizard 127.0.0.1 5000 voldemort
[WIZARD voldemort] CONNECTED server=127.0.0.1:5000
[WIZARD voldemort] RECEIVED ERR HOGWARTS_FULL
[WIZARD voldemort] DISCONNECTED reason=HOGWARTS_FULL
fatih@fatih:~/Desktop/230104004090_hw6$
```

## Professor Snape Terminal (Rejected)



```
fatih@fatih:~/Desktop/230104004090_hw6$ ./professor 127.0.0.1 5000 snape
[PROFESSOR snape] CONNECTED server=127.0.0.1:5000
[PROFESSOR snape] RECEIVED ERR HOGWARTS_FULL
[PROFESSOR snape] DISCONNECTED reason=HOGWARTS_FULL
fatih@fatih:~/Desktop/230104004090_hw6$
```

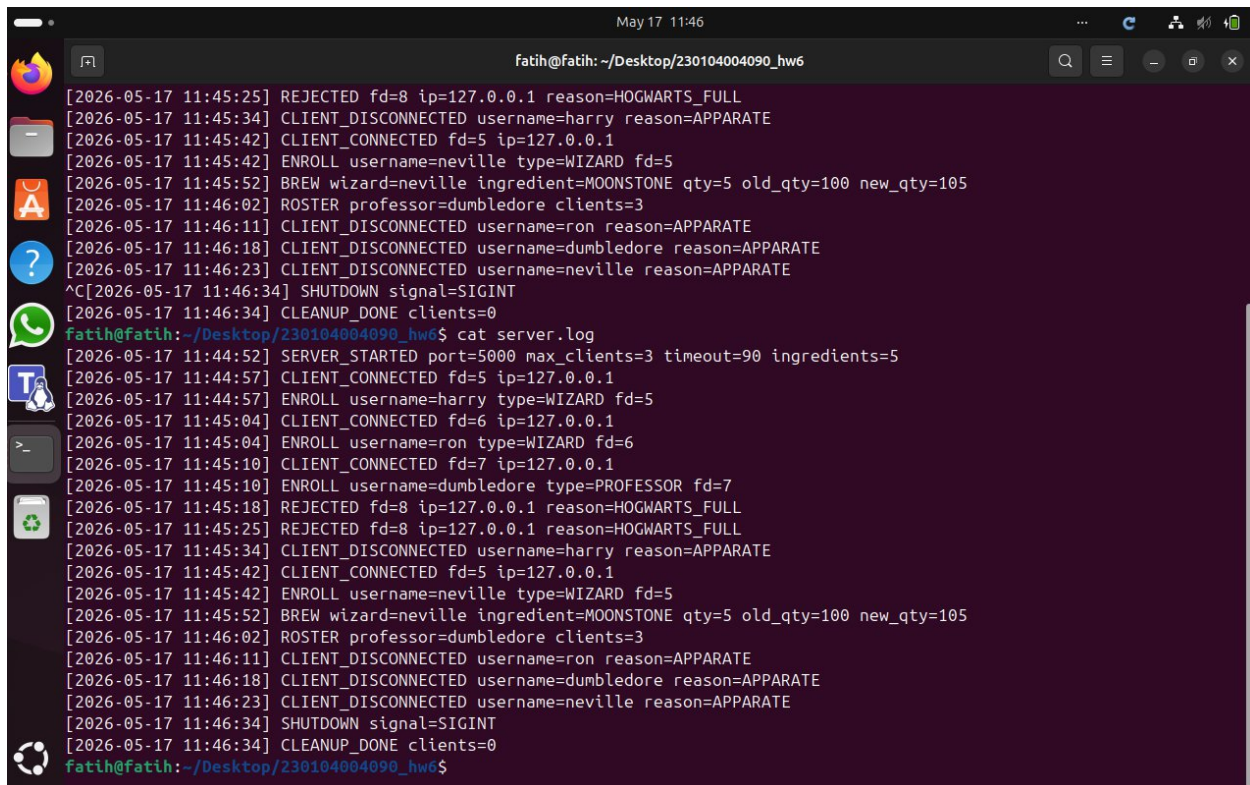
## Wizard neville Terminal



```
fatih@fatih:~/Desktop/230104004090_hw6$ ./wizard 127.0.0.1 5000 neville
[WIZARD neville] CONNECTED server=127.0.0.1:5000
[WIZARD neville] RECEIVED OK ENROLL neville
BREW MOONSTONE 5
[WIZARD neville] SENT BREW MOONSTONE 5
[WIZARD neville] RECEIVED OK BREW MOONSTONE 5 105
^C[WIZARD neville] SENT APPARATE
[WIZARD neville] RECEIVED OK APPARATE
[WIZARD neville] DISCONNECTED reason=APPARATE
fatih@fatih:~/Desktop/230104004090_hw6$
```

*Explanation: The screenshots show that the server rejects new connections with ERR HOGWARTS\_FULL once the maximum number of clients is reached, that a slot is recovered when an existing client disconnects, and that a new client (neville) can then connect and operate normally. The ROSTER reply lists exactly the clients that are connected at that moment.*

Note: Timeout was set to 90 s (instead of 60 s) only for manual convenience; it does not change the capacity logic shown here.



```
fatih@fatih: ~/Desktop/230104004090_hw6
[2026-05-17 11:45:25] REJECTED fd=8 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-17 11:45:34] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-17 11:45:42] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:45:42] ENROLL username=neville type=WIZARD fd=5
[2026-05-17 11:45:52] BREW wizard=neville ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-17 11:46:02] ROSTER professor=dumbledore clients=3
[2026-05-17 11:46:11] CLIENT_DISCONNECTED username=ron reason=APPARATE
[2026-05-17 11:46:18] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-17 11:46:23] CLIENT_DISCONNECTED username=neville reason=APPARATE
^C[2026-05-17 11:46:34] SHUTDOWN signal=SIGINT
[2026-05-17 11:46:34] CLEANUP_DONE clients=0
fatih@fatih:~/Desktop/230104004090_hw6$ cat server.log
[2026-05-17 11:44:52] SERVER_STARTED port=5000 max_clients=3 timeout=90 ingredients=5
[2026-05-17 11:44:57] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:44:57] ENROLL username=harry type=WIZARD fd=5
[2026-05-17 11:45:04] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-17 11:45:04] ENROLL username=ron type=WIZARD fd=6
[2026-05-17 11:45:10] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-17 11:45:10] ENROLL username=dumbledore type=PROFESSOR fd=7
[2026-05-17 11:45:18] REJECTED fd=8 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-17 11:45:25] REJECTED fd=8 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-17 11:45:34] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-17 11:45:42] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:45:42] ENROLL username=neville type=WIZARD fd=5
[2026-05-17 11:45:52] BREW wizard=neville ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-17 11:46:02] ROSTER professor=dumbledore clients=3
[2026-05-17 11:46:11] CLIENT_DISCONNECTED username=ron reason=APPARATE
[2026-05-17 11:46:18] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-17 11:46:23] CLIENT_DISCONNECTED username=neville reason=APPARATE
[2026-05-17 11:46:34] SHUTDOWN signal=SIGINT
[2026-05-17 11:46:34] CLEANUP_DONE clients=0
fatih@fatih:~/Desktop/230104004090_hw6$
```

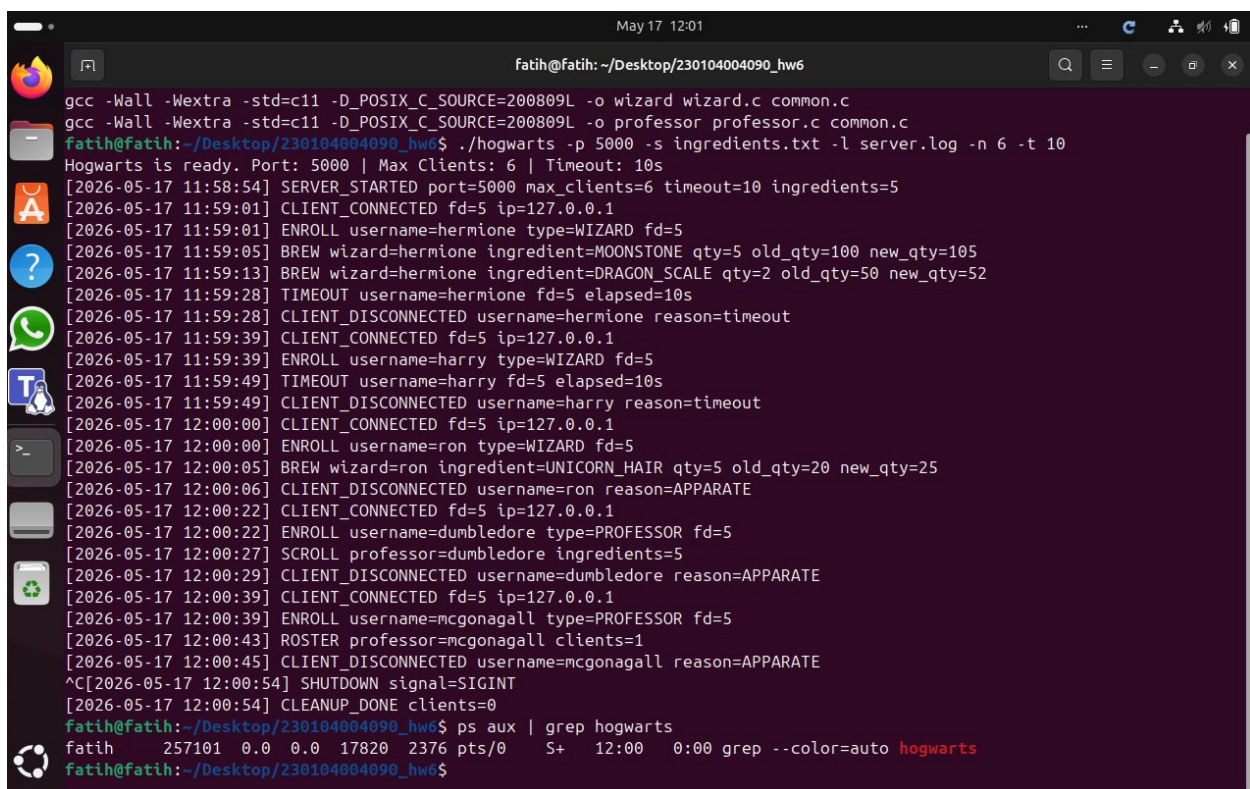
## 7.4 Scenario 4

Command:

```
./hogwarts -p 5000 -s ingredients.txt -l server.log -n 6 -t 10
```

Steps performed: wizard hermione and wizard harry connect. hermione runs BREW MOONSTONE 5, SPELLBOOK, BREW DRAGON\_SCALE 2. harry sends no command at all and is automatically disconnected after the 10-second idle timeout. New clients wizard ron, professorumbledore, and professor mcgonagall then connect and run BREW UNICORN\_HAIR 5, SCROLL, and ROSTER respectively. Finally Ctrl+C is sent to the server terminal, ps aux | grep hogwarts is run to show no leftover process, and cat server.log is run.

### Server Terminal



```
fatih@fatih: ~/Desktop/230104004090_hw6
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o wizard wizard.c common.c
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o professor professor.c common.c
fatih@fatih:~/Desktop/230104004090_hw6$ ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 6 -t 10
Hogwarts is ready. Port: 5000 | Max Clients: 6 | Timeout: 10s
[2026-05-17 11:58:54] SERVER_STARTED port=5000 max_clients=6 timeout=10 ingredients=5
[2026-05-17 11:59:01] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:59:01] ENROLL username=hermione type=WIZARD fd=5
[2026-05-17 11:59:05] BREW wizard=hermione ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-17 11:59:13] BREW wizard=hermione ingredient=DRAGON_SCALE qty=2 old_qty=50 new_qty=52
[2026-05-17 11:59:28] TIMEOUT username=hermione fd=5 elapsed=10s
[2026-05-17 11:59:28] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-17 11:59:39] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:59:39] ENROLL username=harry type=WIZARD fd=5
[2026-05-17 11:59:49] TIMEOUT username=harry fd=5 elapsed=10s
[2026-05-17 11:59:49] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-17 12:00:00] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 12:00:00] ENROLL username=ron type=WIZARD fd=5
[2026-05-17 12:00:05] BREW wizard=ron ingredient=UNICORN_HAIR qty=5 old_qty=20 new_qty=25
[2026-05-17 12:00:06] CLIENT_DISCONNECTED username=ron reason=APPARATE
[2026-05-17 12:00:22] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 12:00:22] ENROLL username=dumbledore type=PROFESSOR fd=5
[2026-05-17 12:00:27] SCROLL professor=dumbledore ingredients=5
[2026-05-17 12:00:29] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-17 12:00:39] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 12:00:39] ENROLL username=mcgonagall type=PROFESSOR fd=5
[2026-05-17 12:00:43] ROSTER professor=mcgonagall clients=1
[2026-05-17 12:00:45] CLIENT_DISCONNECTED username=mcgonagall reason=APPARATE
^C[2026-05-17 12:00:54] SHUTDOWN signal=SIGINT
[2026-05-17 12:00:54] CLEANUP_DONE clients=0
fatih@fatih:~/Desktop/230104004090_hw6$ ps aux | grep hogwarts
fatih 257101 0.0 0.0 17820 2376 pts/0 S+ 12:00 0:00 grep --color=auto hogwarts
fatih@fatih:~/Desktop/230104004090_hw6$
```

## Wizard hermione Terminal

```
fatih@fatih: ~/Desktop/230104004090_hw6
fatih@fatih:~/Desktop/230104004090_hw6$ ./wizard 127.0.0.1 5000 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5000
[WIZARD hermione] RECEIVED OK ENROLL hermione
BREW MOONSTONE 5
[WIZARD hermione] SENT BREW MOONSTONE 5
[WIZARD hermione] RECEIVED OK BREW MOONSTONE 5 105
SPELLBOOK
[WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5
BREW DRAGON_SCALE 2
[WIZARD hermione] SENT BREW DRAGON_SCALE 2
[WIZARD hermione] RECEIVED OK BREW DRAGON_SCALE 2 52
SPELLBOOK
[WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5,DRAGON_SCALE:2
[WIZARD hermione] RECEIVED TIMEOUT DISCONNECT
[WIZARD hermione] DISCONNECTED reason=timeout
fatih@fatih:~/Desktop/230104004090_hw6$
```

## Wizard harry Terminal (Idle Timeout)

```
fatih@fatih: ~/Desktop/230104004090_hw6
fatih@fatih:~/Desktop/230104004090_hw6$ ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5000
[WIZARD harry] RECEIVED OK ENROLL harry
[WIZARD harry] RECEIVED TIMEOUT DISCONNECT
[WIZARD harry] DISCONNECTED reason=timeout
fatih@fatih:~/Desktop/230104004090_hw6$
```



## Wizard ron Terminal

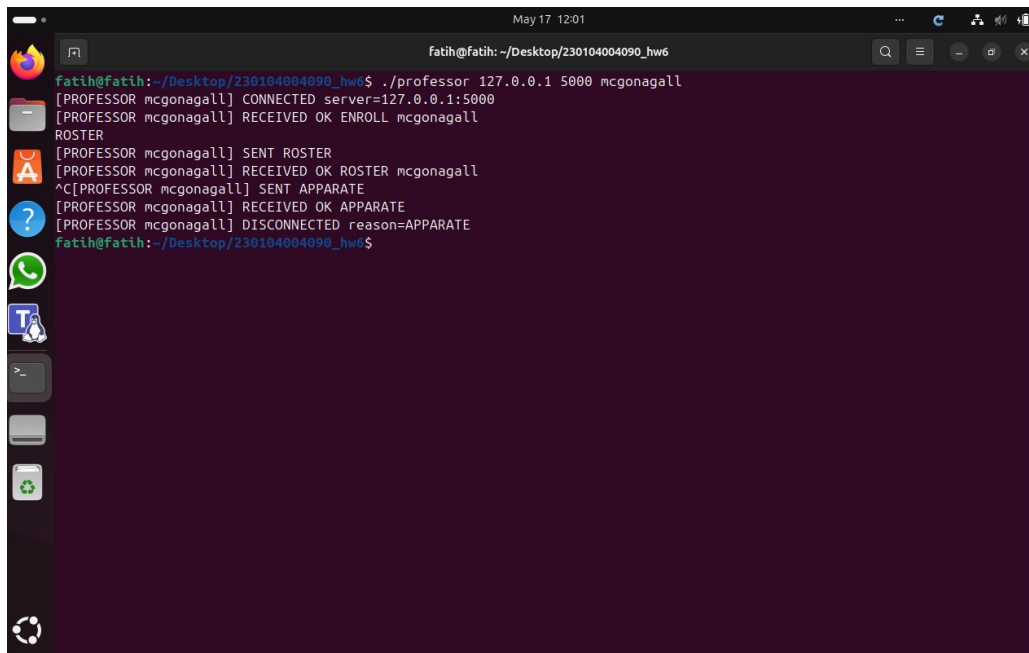
```
fatih@fatih: ~/Desktop/230104004090_hw6
fatih@fatih:~/Desktop/230104004090_hw6$ ./wizard 127.0.0.1 5000 ron
[WIZARD ron] CONNECTED server=127.0.0.1:5000
[WIZARD ron] RECEIVED OK ENROLL ron
BREW UNICORN_HAIR 5
[WIZARD ron] SENT BREW UNICORN_HAIR 5
[WIZARD ron] RECEIVED OK BREW UNICORN_HAIR 5 25
^C[WIZARD ron] SENT APPARATE
[WIZARD ron] RECEIVED OK APPARATE
[WIZARD ron] DISCONNECTED reason=APPARATE
fatih@fatih:~/Desktop/230104004090_hw6$
```

## Professor dumbledore Terminal

```
fatih@fatih: ~/Desktop/230104004090_hw6
fatih@fatih:~/Desktop/230104004090_hw6$ ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5000
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
SCROLL
[PROFESSOR dumbledore] SENT SCROLL
[PROFESSOR dumbledore] RECEIVED OK SCROLL MOONSTONE:105,DRAGON_SCALE:52,PHOENIX_FEATHER:30,MANDRAKE_ROOT:75,UNICORN_HAIR:25
^C[PROFESSOR dumbledore] SENT APPARATE
[PROFESSOR dumbledore] RECEIVED OK APPARATE
[PROFESSOR dumbledore] DISCONNECTED reason=APPARATE
fatih@fatih:~/Desktop/230104004090_hw6$
```

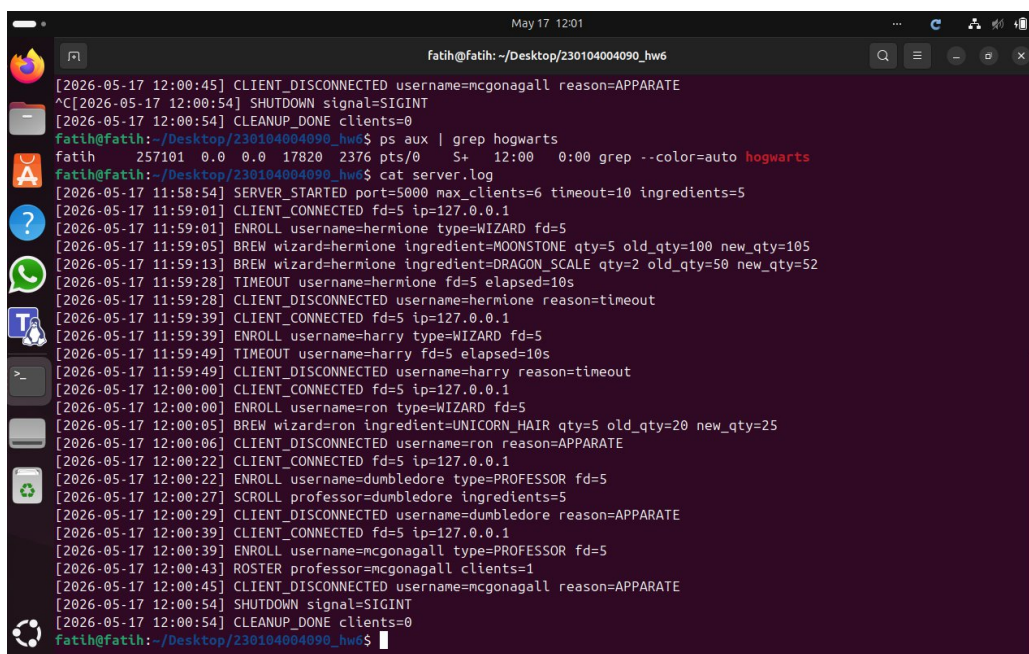


## Professor mcgonagall Terminal



```
fatih@fatih: ~/Desktop/230104004090_hw6
fatih@fatih:~/Desktop/230104004090_hw6$ ./professor 127.0.0.1 5000 mcgonagall
[PROFESSOR mcgonagall] CONNECTED server=127.0.0.1:5000
[PROFESSOR mcgonagall] RECEIVED OK ENROLL mcgonagall
ROSTER
[PROFESSOR mcgonagall] SENT ROSTER
[PROFESSOR mcgonagall] RECEIVED OK ROSTER mcgonagall
^C[PROFESSOR mcgonagall] SENT APPARATE
[PROFESSOR mcgonagall] RECEIVED OK APPARATE
[PROFESSOR mcgonagall] DISCONNECTED reason=APPARATE
fatih@fatih:~/Desktop/230104004090_hw6$
```

*Explanation: The screenshots show that a client which sends no command is automatically disconnected after the configured idle timeout, with the server logging TIMEOUT username=harry fd=N elapsed=10s and CLIENT\_DISCONNECTED reason=timeout. The ps aux | grep hogwarts output confirms that no hogwarts process remains after shutdown (only the grep line is shown), proving there are no orphan processes or leaked descriptors. Note: This scenario uses the specified 10 s timeout; the 90 s value in Scenarios 1-3 was only for manual convenience, the mechanism is identical.*



```
fatih@fatih: ~/Desktop/230104004090_hw6
[2026-05-17 12:00:45] CLIENT_DISCONNECTED username=mcgonagall reason=APPARATE
^C[2026-05-17 12:00:54] SHUTDOWN signal=SIGINT
[2026-05-17 12:00:54] CLEANUP_DONE clients=0
fatih@fatih:~/Desktop/230104004090_hw6$ ps aux | grep hogwarts
fatih 257101 0.0 0.0 17820 2376 pts/0 S+ 12:00 0:00 grep --color=auto hogwarts
fatih@fatih:~/Desktop/230104004090_hw6$ cat server.log
[2026-05-17 11:58:54] SERVER_STARTED port=5000 max_clients=6 timeout=10 ingredients=5
[2026-05-17 11:59:01] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:59:01] ENROLL username=hermione type=WIZARD fd=5
[2026-05-17 11:59:05] BREW wizard=hermione ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-17 11:59:13] BREW wizard=hermione ingredient=DRAGON_SCALE qty=2 old_qty=50 new_qty=52
[2026-05-17 11:59:28] TIMEOUT username=hermione fd=5 elapsed=10s
[2026-05-17 11:59:28] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-17 11:59:39] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 11:59:39] ENROLL username=harry type=WIZARD fd=5
[2026-05-17 11:59:49] TIMEOUT username=harry fd=5 elapsed=10s
[2026-05-17 11:59:49] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-17 12:00:00] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 12:00:00] ENROLL username=ron type=WIZARD fd=5
[2026-05-17 12:00:05] BREW wizard=ron ingredient=UNICORN_HAIR qty=5 old_qty=20 new_qty=25
[2026-05-17 12:00:06] CLIENT_DISCONNECTED username=ron reason=APPARATE
[2026-05-17 12:00:22] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 12:00:22] ENROLL username=dumbledore type=PROFESSOR fd=5
[2026-05-17 12:00:27] SCROLL professor=dumbledore ingredients=5
[2026-05-17 12:00:29] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-17 12:00:39] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-17 12:00:39] ENROLL username=mcgonagall type=PROFESSOR fd=5
[2026-05-17 12:00:43] ROSTER professor=mcgonagall clients=1
[2026-05-17 12:00:45] CLIENT_DISCONNECTED username=mcgonagall reason=APPARATE
[2026-05-17 12:00:54] SHUTDOWN signal=SIGINT
[2026-05-17 12:00:54] CLEANUP_DONE clients=0
fatih@fatih:~/Desktop/230104004090_hw6$
```

## 8. Challenges

### 8.1 Recomputing the select() Timeout Without sleep() or Threads

The first challenge was enforcing the idle timeout while keeping the server single-threaded and free of sleep(). A fixed select() timeout would either disconnect clients too late or wake the loop unnecessarily often. The solution was to compute, before every select() call, the earliest moment at which any connected client could time out, and to pass exactly the remaining time until that moment as the select() timeout. When select() returns (either because data arrived or because the timeout elapsed), the loop re-runs the idle sweep, so clients are disconnected at the right time without busy-waiting and without a separate timer thread.

### 8.2 Handling Partial TCP Messages and Clean Disconnection

The second challenge was correctly reassembling commands when TCP delivered partial or merged lines, while also freeing all resources on every kind of disconnection (APPARATE, idle timeout, and abrupt hangup). This was solved with a per-client line buffer that accumulates bytes across reads and only processes complete newline-terminated lines, keeping any leftover partial line for the next read. A single disconnect routine is used for all three disconnection cases; it closes the socket, frees the client record, and clears the table slot, which prevents file-descriptor and memory leaks and also makes the freed capacity immediately reusable for new connections.